ПМ3 Разработка модулей ПО.

РО 3.1 Понимать и применять принципы объектно-ориентированного и асинхронного программирования.

Тема 1. Введение в ООП.

Лекция 6. Классы в JavaScript: инкапсуляция и статика.

Цель занятия:

Научить студентов защищать данные объекта с помощью приватных полей, контролировать доступ к ним через геттеры и сеттеры, а также использовать статические свойства и методы для данных, общих для всего класса.

Учебные вопросы:

1. Публичные и приватные поля. Символ #.
2. Геттеры (get) — контролируемое чтение данных.
3. Сеттеры (set) — контролируемая запись данных.
4. Статические свойства и методы (static).
5. Комплексный пример — класс Student.

1. Публичные и приватные поля. Символ #.

Публичные и приватные свойства

По умолчанию все свойства в классах JavaScript являются **публичными**.

Это означает, что к ним можно получить доступ, прочитать или изменить их значение из любого места кода.

Публичные свойства объявляются напрямую в классе или в конструкторе с помощью **this**. Они доступны для прямого доступа.

```
class User {
  constructor(name) {
    this.name = name; // Публичное свойство
  }
}

const user = new User('Иван');
console.log(user.name); // 'Иван'
user.name = 'Пётр'; // можно изменить
console.log(user.name); // 'Пётр'
```

С 2020 года в JavaScript появилась возможность объявлять **приватные** свойства с помощью символа **решетки #**.

Доступ к таким свойствам ограничен и возможен **только внутри класса**, в котором они были объявлены.

Попытка обратиться к ним извне класса вызовет ошибку. Это обеспечивает инкапсуляцию, скрывая внутреннюю реализацию объекта.

Пример:

```
class Product {
  #price; // Приватное свойство

  constructor(price) {
    this.#price = price;
  }

  getPrice() {
    // внутри класса есть доступ
    return this.#price;
  }
}

const item = new Product(100);
console.log(item.getPrice()); // 100 - Доступ через метод
// вне класса доступа нет
console.log(item.#price);
// SyntaxError: Private field '#price' must be declared in an enclosing class
```

2. Геттеры и сеттеры.

Геттеры (**get**) и сеттеры (**set**) — это специальные методы, которые позволяют контролировать доступ к свойствам, даже к публичным.

Они создают дополнительный уровень абстракции, позволяя добавлять логику при чтении или записи данных, не нарушая инкапсуляцию.

Геттеры (get).

Геттер — это метод, который вызывается, когда вы пытаетесь прочитать значение свойства.

Он объявляется с ключевым словом **get** и используется как обычное свойство, но может выполнять вычисления или форматирование данных.

Назначение: Возвращать вычисляемые значения, которые зависят от других свойств.

Пример:

```
class Person {
  #firstName;
  #lastName;

  constructor(firstName, lastName) {
    this.#firstName = firstName;
    this.#lastName = lastName;
  }

  get fullName() {
    return `${this.#firstName} ${this.#lastName}`;
  }
}

const person = new Person('Мария', 'Иванова');
// К свойствам нельзя обратиться напрямую:
// console.log(person.#firstName); // SyntaxError
// Доступ возможен только через геттер:
console.log(person.fullName); // Мария Иванова
```

Сеттеры (set).

Сеттер — это метод, который вызывается, когда вы пытаетесь изменить значение свойства. Он объявляется с ключевым словом **set** и принимает один аргумент — новое значение.

Назначение: Добавлять логику при записи, например, валидацию данных или их нормализацию.

Пример:

```
class Circle {
  #radius;

  set radius(value) {
    if (value < 0) {
      throw new Error("Радиус не может быть отрицательным.");
    }
    this.#radius = value;
  }
}

const circle = new Circle();
circle.radius = 10; // Вызывается сеттер
// circle.radius = -5; // Бросит ошибку
```

Итог:

Геттеры и сеттеры особенно полезны в сочетании с приватными свойствами, так как они предоставляют контролируемый способ взаимодействия с данными, которые скрыты от внешнего кода.

Таким образом, вы обеспечиваете, что данные, хранящиеся в экземпляре класса, защищены, а внешний код взаимодействует с объектом только через публичный интерфейс, который вы определили.

Это делает ваш код более надёжным и предсказуемым.

3. Статические свойства и методы.

Статические свойства и статические методы в классах JavaScript принадлежат самому классу, а не его отдельным экземплярам.

Это значит, что для доступа к ним не нужно создавать объект (new ClassName()).

Вы обращаетесь к ним напрямую через имя класса. Они полезны, когда у вас есть данные или функции, которые логически связаны с классом, но не зависят от конкретного объекта.

Статические методы.

Статические методы объявляются с использованием ключевого слова **static**.

Они часто используются для вспомогательных функций, которые не требуют доступа к данным экземпляра.

Например, метод для создания объекта из определённого формата данных или для выполнения общей операции, связанной с классом.

Особенность: Статические методы не имеют доступа к **this** экземпляра. В них можно обращаться только к другим статическим методам или свойствам.

Синтаксис:

```
class Calculator {
  // статический метод
  static sum(a, b) {
    return a + b;
  }
}

// Вызов статического метода
const result = Calculator.sum(5, 3);
console.log(result); // Выведет: 8
```

Статические свойства также объявляются с ключевым словом **static** и являются общими для всего класса.

Они полезны для хранения констант или настроек, которые не меняются от экземпляра к экземпляру.

Использование: Часто используются для хранения конфигурационных данных, например, API-адреса, или для подсчёта количества созданных экземпляров.

Синтаксис:

```
class MathConstants {
  // статическое свойство
  static PI = 3.14159;
}

// Доступ к статическому свойству
console.log(MathConstants.PI); // Выведет: 3.14159
```

Пример класса с статическим счётчиком.

В этом примере, статическое свойство **count** инициализируется нулём.

Каждый раз, когда создаётся новый экземпляр класса **Person** через конструктор, мы увеличиваем значение этого свойства на единицу.

Это позволяет нам легко отслеживать, сколько всего объектов **Person** было создано.

```
class Person {
  // Статическое свойство для подсчета экземпляров
  static count = 0;

  constructor(name, age) {
    this.name = name;
    this.age = age;
    // Увеличиваем статический счётчик при создании каждого нового объекта
    Person.count++;
  }
}

// Создание экземпляров
const person1 = new Person('Иван', 30);
const person2 = new Person('Мария', 25);
const person3 = new Person('Анна', 33)
// Доступ к статическому свойству напрямую через класс
console.log(`Общее количество созданных персон: ${Person.count}`);
// Вывод: Общее количество созданных персон: 3
```

Итог:
Статические свойства и методы (static):

- Принадлежат классу, а не его экземплярам.
- Доступ к ним осуществляется напрямую через имя класса (ClassName.method()).
- Полезны для общих функций, констант и счетчиков.
- Объявляются с ключевым словом static.
- Существуют в единственном экземпляре для всего класса.

5. Примеры использования классов.

Представим, что нам нужно создать простую систему для управления данными о студентах.

Класс **Student** поможет нам структурировать информацию о каждом студенте (имя, курс, оценки) и предоставит методы для работы с этими данными.

```
class Student {
  // Приватные свойства для инкапсуляции
  #name;
  #grades = [];

  // Публичное свойство
  course;

  // Статическое свойство, общее для всех экземпляров
  static studentCount = 0;

  constructor(name, course) {
    this.#name = name;
    this.course = course;

    // Увеличиваем общий счетчик студентов при создании нового экземпляра
    Student.studentCount++;
  }
```

```
// Сеттер для добавления оценок
set addGrade(grade) {
  if (grade >= 0 && grade <= 100) {
    this.#grades.push(grade);
  } else {
    console.error("Оценка должна быть в диапазоне от 0 до 100.");
  }
}

// Геттер для вычисления среднего балла
get averageGrade() {
  if (this.#grades.length === 0) {
    return 0;
  }
  const sum = this.#grades.reduce((total, grade) => total + grade, 0);
  return sum / this.#grades.length;
}
```

```
  // Геттер для получения имени студента
  get name() {
    return this.#name;
  }

  // Статический метод для получения общего количества студентов
  static getStudentsCount() {
    return Student.studentCount;
  }
}
```

Использование класса Student:

```
// Создаем двух студентов
const student1 = new Student('Иван Иванов', 2);
const student2 = new Student('Мария Петрова', 3);

// Добавляем оценки для Ивана через сеттер
student1.addGrade = 85;
student1.addGrade = 90;
student1.addGrade = 78;
student1.addGrade = 110; // Это вызовет ошибку в консоли

// Добавляем оценки для Марии через сеттер
student2.addGrade = 95;
student2.addGrade = 88;
student2.addGrade = 55;
student2.addGrade = 70;
```

```
// Получаем и выводим средний балл каждого студента с помощью геттера
console.log(`Средний балл ${student1.name}: ${student1.averageGrade.toFixed(2)}`);
console.log(`Средний балл ${student2.name}: ${student2.averageGrade.toFixed(2)}`);

// Доступ к публичному свойству
console.log(`Курс Ивана: ${student1.course}`);

// Используем статический метод, чтобы узнать общее количество студентов
console.log(`Всего студентов в системе: ${Student.getStudentsCount()}`);
```

```
▶ Оценка должна быть в диапазоне от 0 до 100.
Средний балл Иван Иванов: 84.33
Средний балл Мария Петрова: 77.00
Курс Ивана: 2
Всего студентов в системе: 2
```

Итоги лекции:

Публичные свойства доступны для чтения и записи из любого места.

Приватные свойства (обозначаются #) доступны только внутри класса, обеспечивая инкапсуляцию.

Геттеры (get) и сеттеры (set) — это специальные методы, которые контролируют доступ к свойствам объекта.

Статические свойства и статические методы принадлежат самому классу, а не его экземплярам. Чтобы получить к ним доступ, не нужно создавать объект.

Контрольные вопросы:

- Чем приватное поле отличается от публичного? Как объявить приватное поле в JavaScript?
- Что произойдёт если попытаться обратиться к приватному полю снаружи класса?
- Чем геттер отличается от обычного метода? Как его вызвать?
- Можно ли у геттера быть параметрами? Почему?
- Зачем нужен сеттер, если можно просто присвоить значение напрямую через =?
- Сколько параметров принимает сеттер? Почему не больше?
- Что такое статический метод? Как к нему обратиться?
- Можно ли вызвать статический метод через экземпляр класса?
- В чём разница между student.name и Student.studentCount?
- Для чего используют статический счётчик экземпляров? Где в конструкторе его нужно увеличивать?

Домашнее задание:

https://ru.hexlet.io/programs/js-oop

Материалы лекций:

https://github.com/ShViktor72/Education2025

Обратная связь:

colledge20education23@gmail.com